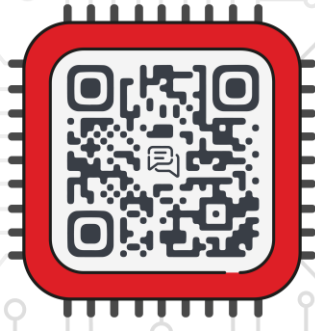




6



15



1700 sp

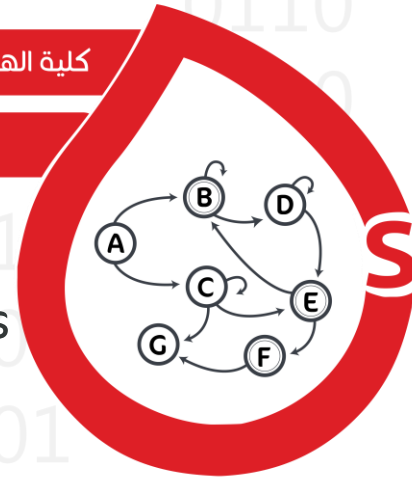
نظري

كلية الهندسة المعلوماتية

السنة الثالثة

Equivalence and Minimization of DFAs

د. هبة حاتم



RB Informatics; 15/05/2025

اللغات الصورية

سنحاول من فكرة تساوي DFAs، بفكرة بناء Minimized DFA.

فكرة الـ Minimization: بناء DFA مصغر عن الـ Original DFA، حيث أنه على مستوى الـ Software والـ Hardware مختصر الحالات ذو أداء أعلى.

Minimization of DFA:

عملية تصغير الـ DFA: هي عبارة عن عملية تقليل عدد الحالات، وبنفس الوقت المحافظة على اللغة التي يدركها الـ DFA (الهدف).

الهدف الأساسي: عمل أصغر DFA ممكن بحيث أن يكون مكافئ للـ DFA الأصل (الذي نملكه).

الحالات المتكافئة (Equivalent States)

من أجل حالتين q_0, q_1 : عالجنا كلمة ولتكن w من أجل كلمة ولتكن w ، يجب أن نصل لـ:
إما w مقبولة عند q_0, q_1 معاً أو w مرفوضة عند q_0, q_1 معاً

وبالتالي:

We can say q_0, q_1 are equivalent states

- هذا يعني أي كان الـ String الذي تتم معالجته، فإن سلوك الأوتوماتا يبقى ثابتاً انطلاقاً إما من الحالة الأولى أو الثانية لا تتغير
- الفائدة: دائماً بالـ DFA عندما يكون لـ حالتين متكافئتين، أستطيع دمجهما بحالة واحدة، أي أستطيع تقليل عدد الحالات بالـ DFA



الحالات التي يمكن تفريقها عن بعضها Distinguishable States

كيف تختلف الـ Distinguishable عن الـ Equivalent States

عندما يكون لدي حالتان Distinguishable لا يمكن دمجها لأن عملية الدمج ستجعلني أفقد State لأبد من وجودها وبالتالي سيتغير سلوك الأوتوماتا، لن يستطيع الـ Automaton إدراك نفس اللغة التي كان يدركها من قبل.

كيف أحصل على Two Distinguishable States؟

من أجل حالتين p, q :

1- إذا كانت p is an accepted state و q is not an accepted state أو العكس (أي حالة نهائية والأخرى لا)

2- من أجل $x \in \Sigma$ كان:

يجب أن يعطي الانتقال $\delta(p, x)$ $\delta(q, x)$ حالات مميزة

ومنه تكون p, q حالات مميزة distinct

$\delta(p, x)$: الحالة التي ينتقل لها P عندما يدخلها x

إذن ما هو تعريف الـ Minimization of DFA؟

تقليل عدد الحالات من أجل تحسين الأداء وبالتالي تحسين السرعة

Example: DFA with 5 states $\rightarrow$ DFA with 4 states

يمكننا بناء / عمل minimization من خلال دمج عدد من الحالات شرط أن تكون هذه الحالات متكافئة

ملاحظة: لا نستطيع دمج حالات مميزة distinguished states وإنما ندمج Equivalent States فقط

قواعد التكافؤ Rules of Equivalency

من أجل حالة A or B قمت بإدخال كلمة (word) " x " وليس محرف، فإن الانتقال δ :

$$\begin{array}{ll} \delta(A, x) \rightarrow F & \delta(A, x) \not\rightarrow F \\ \delta(B, x) \rightarrow F & \delta(B, x) \not\rightarrow F \end{array}$$

Finite States: بنفس الوقت تنتمي لـ F أو بنفس الوقت لا تنتمي لـ F

Minimization Methods

1. Partitioning method (Hopcroft's Algorithm):

مستخدمة فعلياً في الاستخدام التجاري

2. Table-Filling Algorithm (Myhill-Neorde):

خوارزمية ملء الجدول، وهي الخوارزمية الأقدم، تم البدء بدراستها عام 1950، أحد الأشخاص قامو بدراستها لا يزال قائماً على عمله بجامعة بأمريكا، وانتهت (تم تبنيها بالرياضيات) عام 1960، وفي عام 1971 تم وضع الورقة التي أسست لخوارزمية الـ Partitioning Method والتي تم اعتمادها، يعني الأساس الرياضي هو خوارزمية "ملء الجدول" اشتق عنها خوارزمية الـ Partitioning Method والتي يتم استخدامها تجارياً

■ أولاً خوارزمية ال Partitioning Method:

هي خوارزمية كتبها عالم الرياضيات "John Hopcroft" عام 1971 تعتمد على ما يلي:

1- Initialization:

أولاً تبدأ بتقسيم الحالات إلى مجموعتين:

- مجموعة تنتمي لها ال accept states

- مجموعة تنتمي لها ال non-accept states

وتعتبر هاتان المجموعتان أول مجموعتين يمكن بناؤهما بحيث:

- "يستحيل أن يكون عنصر في المجموعة الأولى أن يكون مكافئ لعنصر من المجموعة الثانية"
- وبعدها يبدأ العمل بـ "Refinement" بناءً على دراسة الانتقالات لكل State على كامل أبجدية اللغة
- وبعدها يقارن بالحالات التي تنتمي لنفس المجموعة، ويمكن أن يعيد التقسيم، وسيتم فهم هذا بالمثال.
- تسمى باسم Partitioning أو Hopcroft's Algorithm نسبة للعالم الذي كتبها

خوارزمية ملء الجدول Table-Filling Algorithm

كتبها العالمان (جون مايهل ونيل نيرود) وقاموا بوضع أساسها قبل خوارزمية Hopcroft، نشرت عام 1960 واعتمدت في الرياضيات عام 1960. تعتمد على فكرة بناء جدول، وتمييز الحالات أو عدم تمييزها بناءً على مجموعة من المعايير (سندرسها بعد قليل)

سنبداً بالخوارزمية المعتمدة بالتجارة (على مستوى ال software) Hopcroft's Algorithm والتي تم تقديمها عام 1971، وكان هدف العالم هوبكروفت عندما أنشأ الورقة البحثية والتي تم تبنيها فيما بعد هو تصغير ال DFA بشكل فعال، بحيث أن يعتمد على فكرة التقسيم وأن يتبنى فكرة ال Equivalence على مستوى ال Classes. هوبكروفت حصل على جائزة Turing Machine على إنشائه لهذه الخوارزمية والتي كانت نسخة مطورة من خوارزمية Table-Filling.

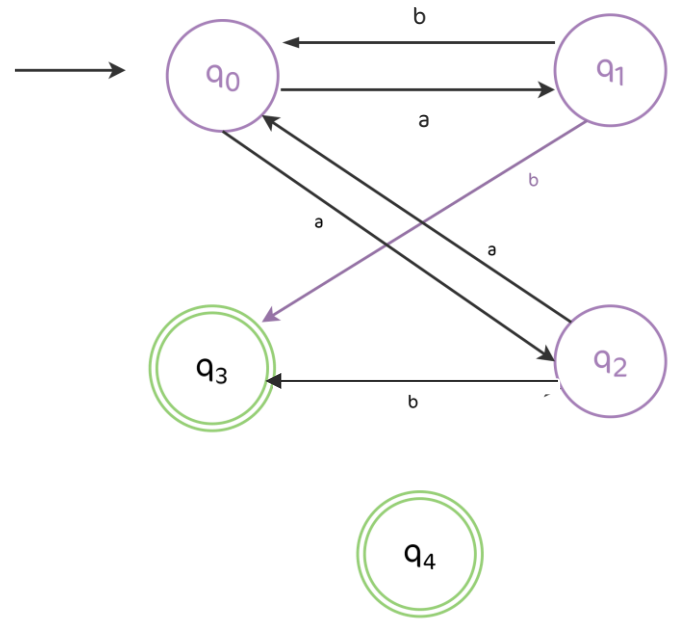
Class of Equivalence: يعني أنا عندي مجموعتين، الأولى تحوي مجموعة من Equivalent States بمرحلة والثانية تحوي مجموعة من ال Equivalent States بمرحلة ثانية، وبعد ذلك أقوم بعمل Refinement (تكرار) بناءً على دراسة انتقال كل حالة والحالة التي تليها. ثم أقوم بدراسة كل حالة من الحالات التي تنتمي لنفس المجموعة عندما تأخذ input Symbol. ثم نقوم بمقارنة هاتين الحالتين هل تنتميان لنفس المجموعة أم لا؟

تنتميان لنفس المجموعة ← إذاً الحالتان متكافئتان

لا تنتميان لنفس المجموعة ← يعيد التقسيم

ليكن لدينا المثال التالي "Dummy" يعني الـ DFA لا على التعيين حتى لو فيه عدد قليل من الحالات:

Dummy تعني اخترعناه الآن وليس كاملاً
(لا يمكن أن يكون بدون انتقالات)



1. 0-Eq

الخطوة الأولى:

وفيه الخوارزمية تقسم الـ States إلى مجموعتين، مجموعة تحتوي الـ Accepting States ومجموعة تحتوي الـ Non-Accepting States وبناءً على ذلك في المثال:

مجموعة الحالات غير المقبولة: $\{q_0, q_1, q_2\}$

مجموعة الحالات المقبولة: $\{q_3, q_4\}$

وهاتان المجموعتان يستحيل أن تكافئا بعضهما، أي:

$$q_0 \not\approx q_3$$

$$q_1 \not\approx q_3, \dots$$

ولكن ضمن المجموعة $\{q_0, q_1, q_2\}$ قد يمكنني الحصول على حالات متكافئة فيما بعد، والحالات المتكافئة منها فيما بعد سأقوم بجمعهم بنفس الحالة ومن ثم بناء DFA Minimization

عناصر المجموعة $\{q_0, q_1, q_2\}$ اسمها 0-Equivalent

أي q_0, q_1, q_2 هي حالات متكافئة على المستوى صفر (Zero)

والمستوى (Zero): هو المستوى الذي تكون فيه الكلمة طولها صفر لأننا لم ندرس أي انتقال بعد.
الآن:

نقول عن p, q حالات متكافئة، انطلاقاً من p, q تكون w إما rejected أو accepted، وهنا w طولها zero لم تأخذ شيء $\leftarrow q_0, q_1, q_2$ هي 0-eq

2. 1-eq:

من أجل كل حالة من المجموعة الواحدة، يجب أن أتأكد فيما إذا كانت الحالات لا تزال متكافئة مع بعضها البعض أم لا، كيف؟

نقوم بدراسة كل حالة من الحالات انتقالاتها عند كل عنصر من عناصر أبجدية اللغة

q_0 أدرس انتقالها عند a, b حيث $\Sigma = \{a, b\}$ وأيضاً q_1

هل q_0, q_2 تنتميان لنفس المجموعة؟

$$\delta(q_0, a) = q_2$$

$$\delta(q_1, a) = q_0$$

نعم، أدرس الانتقال عند b

$$\delta(q_0, b) = q_1$$

$$\delta(q_1, b) = q_3$$

q_3, q_1 لا تنتميان لنفس المجموعة

q_1, q_0 حالتان غير متكافئتان، وبالتالي ستنتهيان لمجموعات مختلفة

نفس المجموعة $q_0, q_1 \notin$

ندرس q_2 ونقارن مع q_1, q_0

لاحظ، في حال كانت النتيجة q_1, q_0 متكافئتان عندها يكفي أن نقارن q_3 مع أحدهما، أي مع q_0 أو q_1

لكن هذا غير محقق، فسنقارن مع q_0 و q_1

- مع q_2 :

نلاحظ عند دراسة الانتقال عند b :

$$\delta(q_2, a) = q_0$$

$$\delta(q_2, b) = q_3$$

q_1, q_3 لا تنتميان لنفس المجموعة أما عند a فهما تنتميان لنفس المجموعة

وبالتالي q_0, q_2 غير متكافئتين وتنتميان لمجموعات مختلفة

q_2 مع q_1 :

نجد أن النتيجة عند دراسة الانتقال عند رموز اللغة هي نفسها عند q_1, q_2 (حيث q_3, q_0)

بالتالي q_2, q_1 متكافئتان وتنتميان لنفس المجموعة، فتكون النتيجة:

$\{q_0\}$,

$\{q_1, q_2\}$,

$\{q_3, q_4\}$

الآن: يجب أن نقارن عناصر المجموعة الثانية مع بعضها، أي q_3 مع q_4 لننظر هل هما متكافئتان أم لا ولكن لأن هذا المثال dummy لن نكملة وسنتقل لمثال حقيقي.

هذه المرحلة اسمها (Refinement) التحسين أو التكرير:

حيث هذه الخوارزمية تقوم (iteratively) بشكل تكراري بتحسين هذه المجموعات (وهي مجموعتان تم ذكرهما سابقاً، مجموعة الحالات النهائية ومجموعة الحالات غير النهائية) بناءً على الانتقالات لكل رمز من رموز الإدخال.

- لكل مجموعة، يتم التحقق مما إذا كان بالإمكان تقسيمها إلى مجموعات أصغر بناءً على الانتقالات. درجة تعقيد هذه الخوارزمية: $O(n \log n)$
- وهذه الخوارزمية أسرع بكثير من خوارزمية الـ Table Method ما يجعلها مثالية لتقليل DFA عندما يكون عدد الحالات كبير حيث أنها تعتمد على التجزئة.

Types of Equivalences

0-Equivalence:

تقسم الحالات لمجموعتين:

- 1- مجموعة الحالات المقبولة: وفيها تكون الحالات متكافئة من النوع 0-Eq فقط لكن بالمرحلة اللاحقة (1-Eq) قد لا يكونوا متكافئين.
- 2- مجموعة الحالات غير المقبولة.

1-Equivalence:

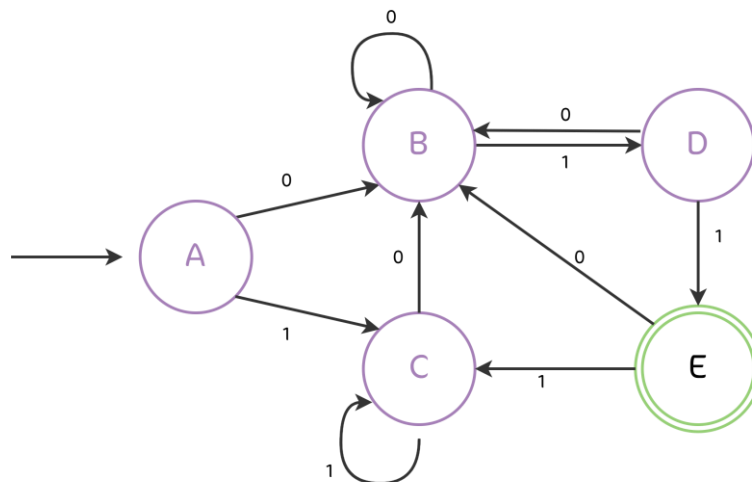
ندرس انتقال كل حالة عند كل حرف من محارف اللغة، وكذلك الأمر لكل الحالات الأخرى التي تنتمي لنفس المجموعة، مثلاً:

$$\begin{array}{lcl} \delta(X, a) = Y & : & W, Y \in \text{نفس المجموعة} \\ \delta(Z, a) = W & & X, Z \text{ are equivalence} \end{array}$$

General Concept:

- تستمر العملية بمستويات أعلى من التكافؤ (مثل التكافؤ من الدرجة الثالثة الخ...) حيث تعتبر الحالات متكافئة إذا حافظت على التكافؤ عبر المزيد من الانتقالات.
- تهدف خوارزمية "Hopcroft" بشكل أساسي إلى إيجاد الحد الأدنى من مجموعة الحالات من خلال تحسين (تنقية) هذه العلاقات التكافؤية إلى أن نصل إلى عدم إمكانية إجراء أي تقسيم إضافي.

Hopcraft's Algorithm (partitioning method) Ex-1:



$$\begin{aligned}
 0 - Eq: \{A, B, C, D\} \\
 1 - Eq: \delta(A, 0) = B \\
 \delta(B, 0) = B
 \end{aligned}$$

$$\begin{aligned}
 \{E\} \\
 \delta(A, 1) = C \\
 \delta(B, 1) = D
 \end{aligned}$$

بالتالي A, B بنفس المجموعة بالـ 1-Eq ;

نقارن C مع A أو B لأنهما بنفس المجموعة

$$\delta(C, 0) = B$$

$$\delta(C, 1) = C$$

A, B, C بالـ 1-Eq لا يزالوا متكافئين لبعض

$$\delta(D, 0) = B$$

$$\delta(D, 1) = E$$

D تنتمي لمجموعة أخرى، نتوقف عن مقارنة D مع A, B, C لأنهم ينتموا لمجموعات مختلفة.

في حال لدينا أكثر من حالة Accepted أيضاً سأقوم بمقارنتهم مع بعضهم (تعامل معاملة أي مجموعة أخرى)

2-Eq

(بما أن D خرجت بمجموعة جديدة سنقوم بمقارنة المجموعات على أساسها)

بمقارنة B مع A (الذي قمنا به سابقاً) نجد أن:

A, B لا ينتميان لنفس المجموعة وبالتالي A, B غير متكافئين.

نقارن C مع A:

في حال كانت C تكافئ A أتوقف عن مقارنتها بـ B، في حال لم تكافئها أقارنها مع B حتماً وبالتالي هناك غاية من مقارنة C مع كل من A, B (لأنهما غير متكافئين)

C مع A:

نجد أن A, C متكافئتان (وبإمكاننا التحقق من عدم تكافؤهما مع B بالمقارنة)

لاحظ: أننا وصلنا لمرحلة فيها الانتقالات لـ A و C عند أبجدية اللغة هي نفسها:

$$\delta(A, 0) = B$$

$$\delta(A, 1) = C$$

$$\delta(C, 0) = B$$

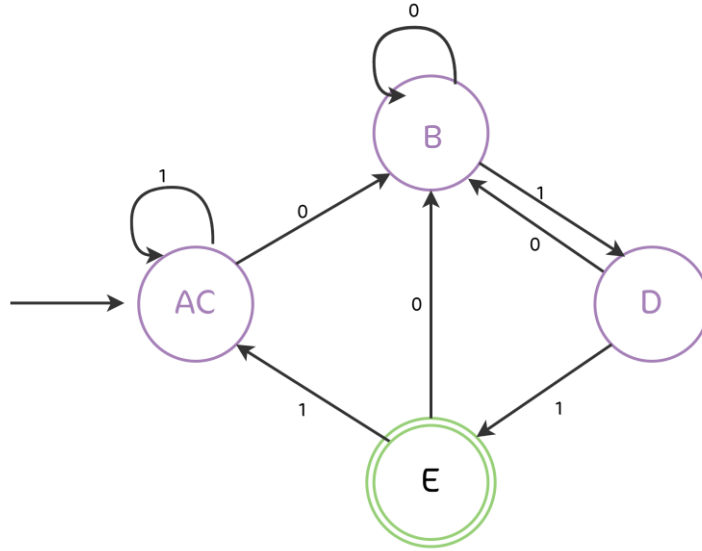
$$\delta(C, 1) = C$$

- هذه هي الحالة النهائية

- A, C هما Equivalent

- الآن نستطيع دمج A, C بحالة واحدة [AC]، وبما أن A هي Start State إذاً AC هي Start State أيضاً، والحالات أصبحت عندي كالتالي:

$$\{AC, B, E, D\}$$



Minimized Finite Automaton of DFA

وهكذا استطعنا الحصول على نسخة أخرى من الـ DFA بعدد حالات أقل من عدد الحالات الأوتومات الأساسي، مما يعطيني أداءً عالياً بدرجة تعقيد $O(n \log n)$ والتي هي محسنة عن الخوارزمية الأساسية وعن الخوارزمية التي سنقوم بدراستها بعد قليل.

شرح مفصل عن الحالات وانتقالها قبل وبعد:

	0	1
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C

$\{ A, B, C, D, E \}$

	0	1
AC	$\{B \cup B\}: B$	$\{C \cup C\}: AC$
B	B	D
C	B	E
E	B	AC

$\{ AC, B, D, E \}$

Table-Filling Method

الجدوى من دراستها: هو دراسة الأساس الرياضي الذي على أساسه بنيت نظرية "Hopcroft" على الرغم من أنه بالاستخدام التجاري "Commercial Use" وعلى مستوى الـ "Software" نستخدم خوارزمية Hopcroft حيث أنها من أجل "Small DFAs" يمكنني استخدامها ولكن من أجل DFA فإن درجة تعقيد الخوارزمية كبير (أسّي) وبالتالي عملية بنائها على مستوى الـ Hardware والـ Software مكلفة جداً، بدأت دراستها عام 1950 ونشرت عام 1960.

Recall:

$$DFA M = (Q, \Sigma, \delta, q_0, F)$$

- Q : مجموعة الحالات set of all states
- Σ : مجموعة كل محارف الأبجدية
- δ : تابع الانتقال
- q_0 : start states
- F : set of all finite states

Formal Definition for Distinct States

1.

$$P \rightarrow F, \quad q \not\rightarrow F \rightarrow q, p \text{ غير متكافئتين}$$

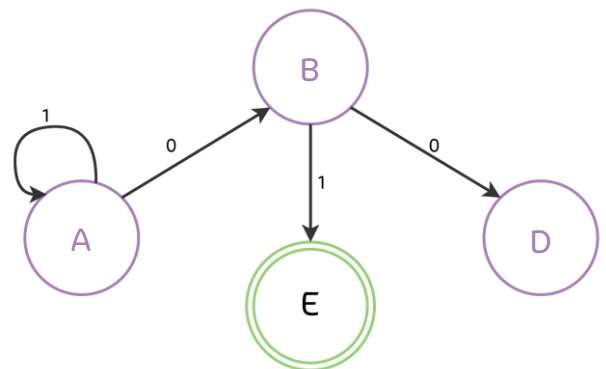
(vice versa) صحيح

2.

من أجل أي $a \in \Sigma$ فإذا كان انتقال $\delta(q, a)$ و $\delta(p, a)$ حالتين غير متكافئتين q, p ليستا متكافئتين
ومن هذا التعريف أخذ العالمان (Myhill - Neorode) الأساس وقاموا بعمل الخوارزمية "Table-Filling" والتي تقول:
Create lower triangular table Distinct, initially blank
سنقوم بعملية اختزال من الأعلى، ونلاحظ من اسم الخوارزمية (خوارزمية ملء الجدول) أي نريد ملء الجدول رويداً رويداً

Let's imagine this dummy example:

A				
B				
D				
E				
	A	B	D	E



أوزع الحالات على الأعمدة والأسطر في الجدول

ولكن كيف تم الاختزال؟؟

Distinct initially blank:

وهي الحالة التي تكون فيها جميع الخلايا فارغة، لكن متى تمتلئ؟

Steps:

1.

Draw a table for all pairs of states (P, Q), and mark all cells with the same state (P, P).

ارسم جدول من أجل كل أزواج الحالات (يتم تشكيل الجدول من أعمدة وأسطر يتم توزيع الحالات عليها)، واطمس كل الخلايا بالحالات المتشابهة مثل (A, A).

2.

Mark all pairs where $p \in F$ and $Q \neq F$ (F: final state)

قم بطمس كل خلية تحتوي على حالة نهائية وحالة غير نهائية.

3.

If there are any unmarked pairs (P, Q) such that $[\delta(p, x), \delta(Q, x)]$ is marked, then mark $[P, Q]$ "where x is an input symbol". Repeat this until no more markings can be made.

بعد ذلك الخلية الفارغة (P, Q) سأقوم بانتقال حالاتها عند رموز اللغة، إذا كانت خلية زوج الانتقال للحالتين وذلك من أجل محرف واحد على الأقل ممثلة، أقوم بعمل mark للخلية (P, Q)، أقوم بتكرار العملية حتى أنهي جميع الخلايا / لا يمكنني عمل mark أكثر.

4.

Combine all the unmarked pairs and mark them single state in the minimized DFA.

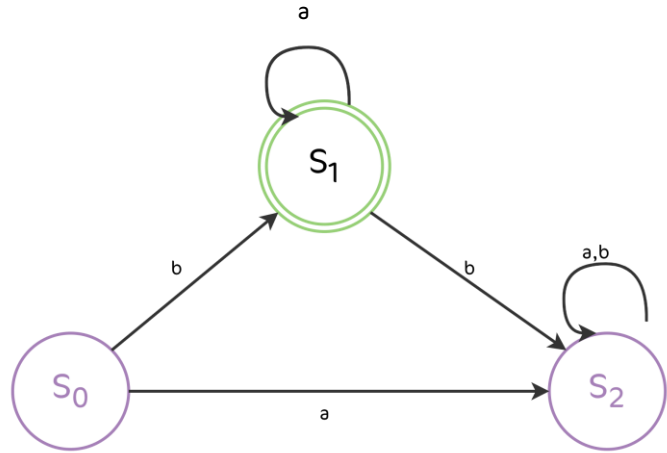
أدمج زوج الحالات للخلية الفارغة بحالة واحدة (P, Q) وأضعهم في minimized DFA

معرفة عملية سير التكرارات مهمة جداً

قد يصعب عليكم استيعاب هذا ولكن مع حل المثال سنفهم كل شيء.



S_0			
S_1	ε		
S_2	ε		
	S_0	S_1	S_2



أغني معلوماتي:

العناصر فوق خط التناظر بالجدول (خط الحالات المتشابهة) تتم مقارنتها مرتين، كما تتم مقارنة العناصر تحت خط التناظر، سنقوم باختزالها حيث أن $(A, B) = (B, A)$ لذلك في كثير من المراجع العلمية سمي الجدول بـ "Lower Triangle Table".

تم اختزال الخلايا التي تحوي حالة نهائية وحالة غير نهائية (s_1, s_0) و (s_2, s_0) وتم اختزال الحالات المتشابهة والخلايا التي من نفس الحالة، والآن لدينا:

$$(s_1, s_0), \quad (s_2, s_0), \quad (s_2, s_1)$$

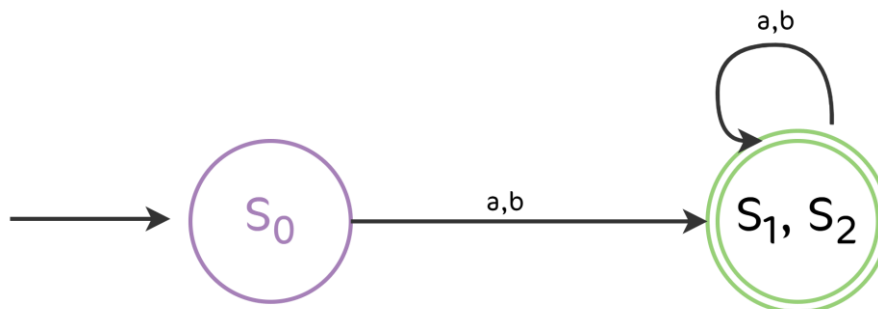
$$(s_2, s_1): \quad \delta(s_2, a) = s_2, \quad \delta(s_1, a) = s_1$$

الآن ننظر للخلية (s_1, s_2) أو (s_2, s_1) نجد أنها فارغة، نذهب للانتقال عند b :

$$\delta(s_2, b) = s_2, \quad \delta(s_1, b) = s_1$$

وهي الخلية (s_2, s_1) نفسها

بالتالي الخلية (s_2, s_1) تبقى فارغة وأستطيع دمج s_2, s_1 في s_1, s_2 الحالة الابتدائية تبقى ابتدائية والحالة النهائية تبقى نهائية



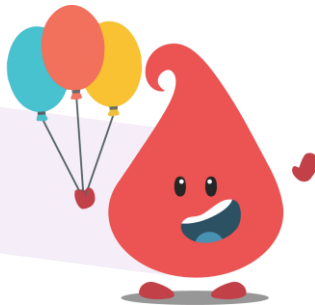
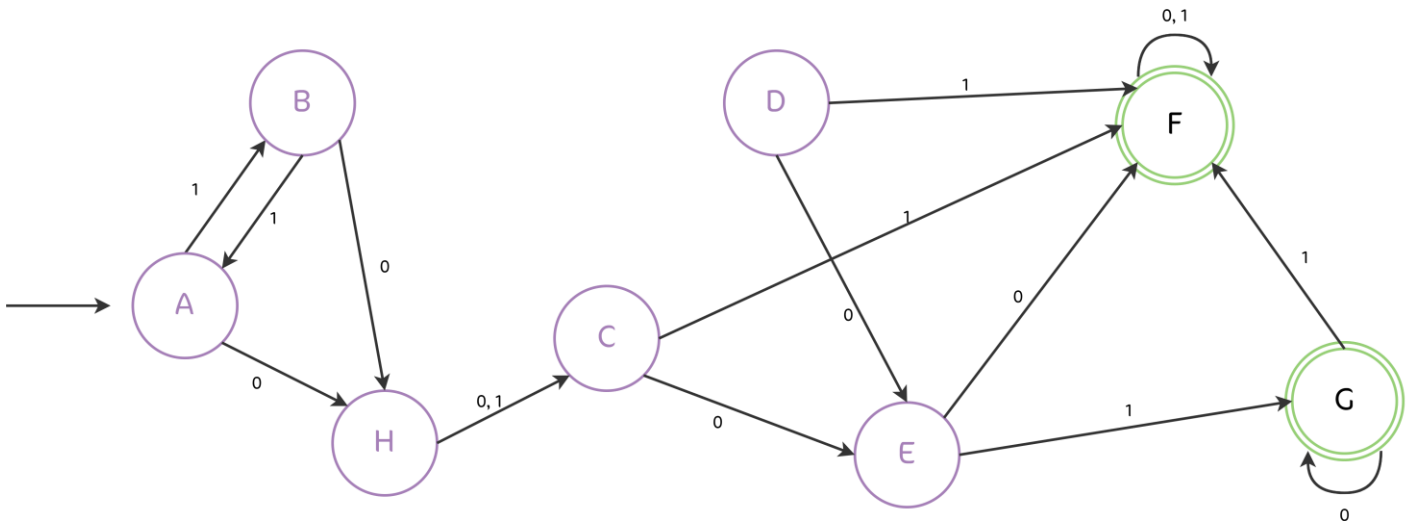
في بعض المرجعيات العليمة نرى الـ labels / الرقع بشكل ϵ أو بشكل $\checkmark$ / $\times$
وبعض المرجعيات اعتمدت التقسيم التالي بوضع العلاقة على الـ labels:

- المرحلة الأولى: حيث يكون بالخلية الواحدة على الأكثر one final state (ونحن نقول حالة نهائية واحدة (distinct) وليس أكثر، لأنهم ليسوا equivalent) يتم التعليم بـ ϵ
- المرحلة الثانية: يتم التعليم بالمحارف التي بينت لنا أن هذه الثنائيات هي distinct (1, 0, ϵ , ...) لكن أنت تقوم بعمل mark كما تحسب وتفهم.

ملاحظة هامة:

في حال وصلت لمرحلة أكون قد علمت كل الخلايا بالجدول تكون هذه الـ DFA الذي حاول عمل minimization مسبقاً هو الـ minimized الـ best case scenario، وهذا مكافئ لأنني وصلت لعدد من الحالات بالـ Hopcroft كل حالة set مافيها غير عنصر واحد.

More complex example:



سأبدأ بالجدول باستثناء الحالات المتماثلة المشكلة للخلايا:

A								
B								
C	1	1						
D	1	1						
E	0	0	0	0				
F	ε	ε	ε	ε	ε			
G	ε	ε	ε	ε	ε			
H	1	1	1	1	0	ε	ε	
	A	B	C	D	E	F	G	H

أول مرور: فيه سأختزل الخلية (نهائية، غير نهائية)

أول مرور نعلم فيه الخلايا التي تحوي حالة نهائية وحالة غير نهائية (تم التعليم بـ ε بالمثل للدلالة على أن هذا هو أول مثال)

(A, B):

$$\delta(A, 0) = H$$

$$\delta(A, 1) = B$$

$$\delta(B, 0) = H$$

$$\delta(B, 1) = A$$

تبقى (AB) فارغة وبالتالي A, B متكافئتان (نضع (B أو A) انتقالات لـ A مباشرة بعد حساب كل الحالات)

(A, C):

$$\delta(C, 0) = E$$

$$\delta(C, 1) = F$$

من هو الرمز الذي كشف أنهم distinct وليس equivalence؟

الـ "1" وبالتالي نعلم الخلية بالرمز "1"

(A, D):

$$\delta(D, 0) = E$$

$$\delta(A, 0) = H$$

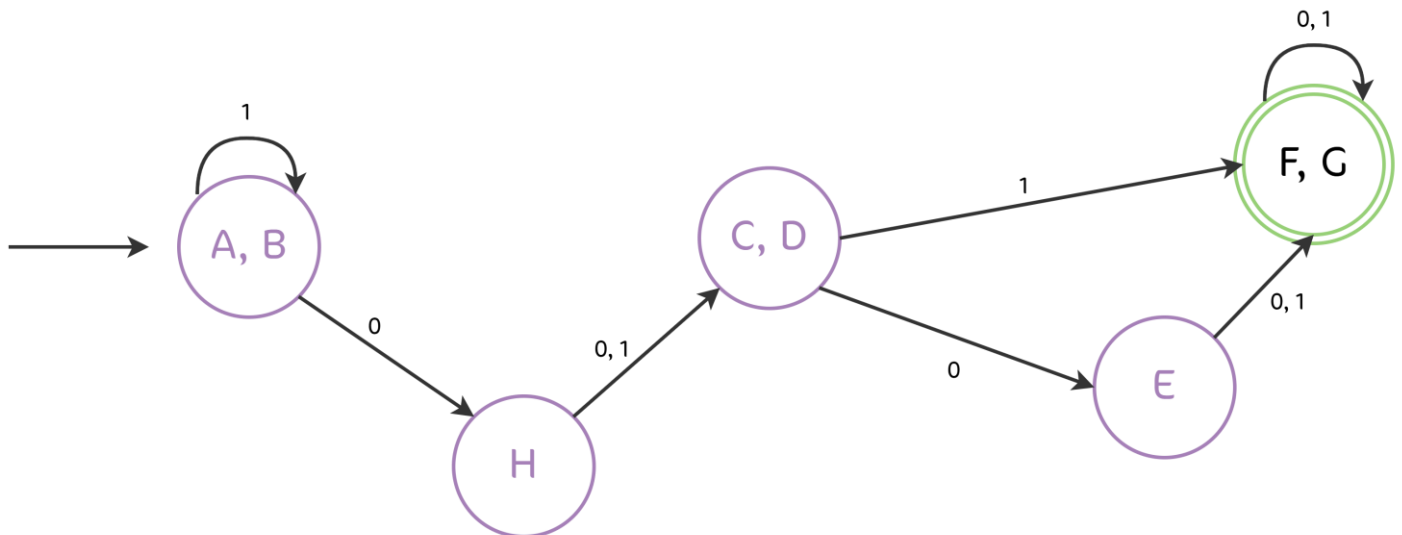
E, H لم يتم تعليمهم بالتالي لا نعلم DA حالياً

$$\delta(D, 1) = E \text{ (نهائية معلمة)}$$

نعلم الـ AD بالرمز "1"

وهكذا أستم بالجدول حتى أصل لمرحلة تكون فيها خلايا معلمة بالـ "1" وخلايا / حقول معلمة بالـ "0" وحقول معلمة بالـ "ε" أو بالـ ✓ أو بالـ ✗ كما أريد.

ويظهر لدي خلايا فارغة، هذه الخلايا الفارغة تحدد ثنائيات يمكن دمجها بحالة واحدة، وبالتالي أحصل على Minimized DFA بثلاثة تكرارات (دمج الحالات المتكافئة من أجل Minimized DFA)



■ الحالات المتكافئة:

-1 A و B

-2 C و D

-3 F و G

هذه أفضل طريقة Subset Construction البناء من خلال جمع المجموعات بدمجهم بدون Union. وبعض المراجع تستبدل الدمج بحالة واحدة فقط من الحالتين.

Hopcroft's Algorithm vs Myhill-Nerode

Based Minimization:

Aspect	Myhill-Nerode Theorem	Hopcroft's Algorithm
Domain	الأساس لا 2015 لا توجد خوارزميات تعتمد على مايهل فقط رياضيات Theoretical (foundational proof)	Practical computational implementational
Use Case	Proving minimality / validity المفهوم الذي تسعى مايهل لإثباته: Two DFAs are equivalence, one of them is minimal for the other	Building minimal DFAs in software بالفعل تستخدم لبناء minimal DFA بالسوفت وير
Complexity Class	بأحسن الحالات ($O(n^2)$) بأسوأ الحالات ($O(K(n^2))$) K عدد محارف أبجدية اللغة	$O(n(\log n))$
Practicality	Impractical beyond small DFAs small DFAs غير فعالة ولكن من أجل فهي فعالة	Efficient for large DFAs
Focus	Equivalence class enumeration	Optimized state partitioning

النهاية

